Lucas–Kanade Incremental Alignment

A step-by-step reading of PDF page 12

Video Processing 0512-4263 — explanatory note

June 22, 2026

PDF page 12 is the *tail end* of the Lucas–Kanade (LK) incremental alignment derivation, and it reads as confusing because the page drops you into the middle of an argument that began on pages 10–11. Below is the whole chain, from the goal to the final least-squares update, with every quantity named.

The goal (page 10)

You have two images. You want a single warp $W(X;P)$ — e.g. translation (2 parameters) or affine (6 parameters) — that makes warped image-2 look like image-1:

$$P^* = \arg\min_P \sum_X \big[\, I_2(W(X;P)) - I_1(X) \,\big]^2.$$

The problem. The parameters P sit *inside* the warp W, so this is non-linear: you would have to search a 6-dimensional space directly. Expensive.

The trick: solve for a small update (page 11)

Do not solve for P all at once. Start from a guess (the identity warp), and on each iteration find a small correction ΔP, then update $P \leftarrow P + \Delta P$. The key move is a first-order **Taylor approximation** of the warped image:

$$I_2\big(W(X;P+\Delta P)\big) \;\approx\; \underbrace{I_2(W(X;P))}_{\text{warp at current } P} \;+\; \underbrace{\nabla I_2 \,\frac{\partial W}{\partial P}}_{\text{brightness change per parameter}} \Delta P.$$

Now ΔP appears **linearly** (outside the warp). That is the entire point: a hard non-linear problem becomes an easy linear one.

The two factors in that middle term are

- $\nabla I_2 = (I_x \;\; I_y)$ — the image gradient (1×2);
- $\dfrac{\partial W}{\partial P}$ — the Jacobian of the warp: how the warped coordinates move when you nudge each parameter. For affine it is 2×6; for translation it is the 2×2 identity.

Multiplying them gives a **row vector, one per pixel**. For affine:

$$\nabla I_2 \,\frac{\partial W}{\partial P} = (\, I_x x \;\; I_x y \;\; I_x \;\; I_y x \;\; I_y y \;\; I_y \,).$$

What page 12 actually does

Step 1 — what is B? Stack every pixel. This is the part the page glosses over. Look at one pixel first. For pixel number i, the row vector $\nabla I_2 \frac{\partial W}{\partial P}$ from the previous step is a single row of 6 numbers (for affine):

$$\text{pixel } i: \qquad b_i = \begin{pmatrix} I_x^{(i)}x_i & I_x^{(i)}y_i & I_x^{(i)} & I_y^{(i)}x_i & I_y^{(i)}y_i & I_y^{(i)} \end{pmatrix},$$

where $I_x^{(i)}, I_y^{(i)}$ are the gradients at that pixel and (x_i, y_i) are its coordinates. **One pixel gives one row.**

Now take *every* pixel in the window — there are N of them — and stack those rows on top of each other. *That* is B:

$$\underbrace{B}_{\text{the matrix}} = \begin{pmatrix} — \, b_1 \, — \\ — \, b_2 \, — \\ \vdots \\ — \, b_N \, — \end{pmatrix} = \begin{pmatrix} I_x^{(1)}x_1 & I_x^{(1)}y_1 & I_x^{(1)} & I_y^{(1)}x_1 & I_y^{(1)}y_1 & I_y^{(1)} \\ I_x^{(2)}x_2 & I_x^{(2)}y_2 & I_x^{(2)} & I_y^{(2)}x_2 & I_y^{(2)}y_2 & I_y^{(2)} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ I_x^{(N)}x_N & I_x^{(N)}y_N & I_x^{(N)} & I_y^{(N)}x_N & I_y^{(N)}y_N & I_y^{(N)} \end{pmatrix}.$$

So B is simply a table: **one row per pixel, one column per parameter.** That is the "$N \times 6$" the page opens with — N rows (pixels), 6 columns (affine parameters). For translation each row has only 2 numbers $\begin{pmatrix} I_x^{(i)} & I_y^{(i)} \end{pmatrix}$, so B is $N \times 2$.

In short: B is the matrix that, when multiplied by the update ΔP (6×1), reproduces the per-pixel linear term $\nabla I_2 \frac{\partial W}{\partial P} \Delta P$ for all N pixels at once.

Step 2 — name the residual I_t (the partner of B). For the same N pixels, define

$$I_t = \begin{pmatrix} I_2(W(X_1;P)) - I_1(X_1) \\ I_2(W(X_2;P)) - I_1(X_2) \\ \vdots \\ I_2(W(X_N;P)) - I_1(X_N) \end{pmatrix}.$$

This is just "warped image-2 minus image-1" evaluated at the *current* guess — the leftover error you are trying to kill. It is an $N \times 1$ column vector (one number per pixel), lined up row-for-row with B.

Dimensions at a glance. B is $N \times 6$, ΔP is 6×1, I_t is $N \times 1$. So $B\Delta P$ is $N \times 1$ and matches I_t pixel-by-pixel — which is exactly what makes the next step a clean least-squares problem.

Step 3 — rewrite the objective compactly. Substituting the Taylor approximation into the goal, each pixel's bracket becomes $I_t + B\Delta P$, so the whole sum of squares is a vector dotted with itself:

$$\Delta P^* = \arg\min_{\Delta P} (I_t + B\Delta P)^T (I_t + B\Delta P).$$

This is plain **linear least squares**, with ΔP the only unknown.

Step 4 — solve it (the normal equations). Expand, differentiate with respect to ΔP, and set to zero:

$$\frac{d}{d\Delta P}\Big[\, I_t^T I_t + 2\,\Delta P^T B^T I_t + \Delta P^T B^T B\,\Delta P \,\Big] = 2B^T I_t + 2B^T B\,\Delta P = 0,$$

$$\Delta P^* = -\,(B^T B)^{-1} B^T I_t.$$

That is the standard least-squares formula — nothing optical-flow-specific yet: it just solves $B\,\Delta P \approx -I_t$ in the least-squares sense.

Step 5 — iterate. Update $P \leftarrow P + \Delta P^*$, re-warp image-2, recompute I_t and B, and solve again. Each pass shrinks the residual and moves you closer to P^*.

The translation special case

Plug $\dfrac{\partial W}{\partial P} = I_{2\times 2}$ in, so $B = (I_x \;\; I_y)$ of size $N \times 2$. Then $B^T B$ and $B^T I_t$ expand into the famous 2×2 system:

$$\Delta P^* = -\begin{pmatrix} I_x^T I_x & I_x^T I_y \\ I_y^T I_x & I_y^T I_y \end{pmatrix}^{-1} \begin{pmatrix} I_x^T I_t \\ I_y^T I_t \end{pmatrix}.$$

The 2×2 matrix on the left is the **structure tensor** (sums of products of gradients over the window). Here $\Delta P = (u, v)$ is the motion.

Why the "crossing point" picture

Each pixel contributes one brightness-constancy equation

$$uI_x + vI_y + I_t = 0.$$

With two unknowns (u, v), **one pixel is one straight line** in the (u, v) plane — not enough to pin down a point. This is the *aperture problem*. Many pixels give many lines; if they all share the same true motion, the lines **all cross at one point**, which is the answer. Real data has noise, so the lines do not cross perfectly — and least squares (Step 4) picks the point closest to all of them.

Page 12 in one sentence. Take the linearized per-pixel equations, stack them into B and I_t, solve the least-squares system $\Delta P^* = -(B^T B)^{-1} B^T I_t$ for the update, add it to P, and repeat.